Advanced Task 1: Legacy Audit and React + TypeScript Migration Architecture

Author: Ahmed · Date: 2026-11-13 · Branch: `docs/react-migration-architecture` · Inputs: this repository (the consolidated vanilla app, Phase 2 Tasks 1–6), the TechSphere API kit 1.1 (`api/techsphere/`), the Lighthouse baseline (`docs/lighthouse/README.md`).

Every number below comes from something you can re-run:

Evidence	Command	Output
Code audit (modules, typing, duplication, state, page weight, tests)	`npm run audit:legacy`	`docs/migration/audit.md`, `audit.json`
Real API responses, 38 calls	`npm run api:capture` (starts the kit; or uses `VITE_API_BASE`)	`docs/migration/observed/*.json`
Field inventory of those responses	`npm run api:fields`	`docs/migration/observed-fields.md`
TypeScript types checked against the responses	`npm run typecheck` (also in CI)	`src/types/api.ts`, `src/types/domain.ts`
Brand tokens as a Tailwind v4 theme	`npm run tokens:tailwind`	`docs/migration/tailwind-theme.css`, `tailwind-tokens.md`

1. Audit of the legacy app

#	Finding	Evidence (audit.md)	What it means for the migration
A1	**No checked types.** There are 70 modules and 5,796 lines, and 0 are TypeScript. 94% of functions have JSDoc, but nothing checks it, and 93 JSDoc types are `{object}`, `{Function}` or `{*}`.	§1: `components/` 53 loose types, `state/` 19, `api/` 18	Port the typed core first. The API boundary is where wrong shapes hurt (A7).
A2	**Duplicated page shell.** The `<header>` (20 lines) and `<footer>` (12 lines) are copied into all 9 HTML pages. They have already drifted into 4 versions each: `index.html` has extra links, and `404.html` and `comparison.html` differ.	§3 table 1	One `<Layout>` with `<Header>`/`<Footer>` removes 288 duplicated lines and the drift.
A3	**Duplicated UI patterns.** Six modules build product `<img>` elements by hand. Eight components format prices at the call site. Seven hand-written live regions announce changes. Ten modules restore focus after a re-render.	§3 table 2	`<ProductImage>`, `<Price>`, one `<Announcer>` and keyed React rendering replace them.
A4	**State is centralised in one wrapper but created many times.** There are 5 storage keys in 2 areas (`brandstore:cart:v1` local, `…saved:v1` local, `…recent:v1`, `…session:v1` and `…flags:v1` session). Every page builds its own store instances: the cart store is created on **9 pages**, the session on 9, and they synchronise through `storage` events. Catalog state lives in the URL. Six components keep module-level `let` state.	§4	In a single-page app there is one `CartProvider` and one `AuthProvider`, and no store per page. Keep the key names and shapes, so legacy and React pages share the cart while both are live (§9).
A5	**No bundler.** Each page downloads 15–34 modules as separate requests (catalog: 34). That is 11–44 KB of JS gzip and 10 KB of CSS gzip, unminified, with no cache-busting. Axios (17.5 KB gzip) is loaded by 2 pages alongside a second, fetch-based HTTP client. Lighthouse mobile performance is 91–98: catalog CLS is 0.143 and cart CLS 0.112, with render-blocking CSS.	§2; `docs/lighthouse/README.md`	Vite bundles, minifies, hashes and splits per route. Drop Axios (one fetch client). React + React DOM + React Router will **raise** first-load JS. The budget in §8 keeps it honest.
A6	**A strong, mostly framework-free test suite.** There are 162 unit tests, 21 API integration tests, 26 jsdom + MSW tests, 9 Playwright specs and 28 journeys. Unit tests cover `state/` at 99.86% and `utils/` at 99.00% of lines.	§5; CI on PR #3	The pure modules port to TypeScript with their tests unchanged: lowest-risk wave. The e2e specs use roles and labels, so they survive a new DOM.
A7	**Messy API data** (observed-fields.md). `price` is a number or a string. `stock_status` is missing on 2 of 24 products. Products page by `page` but orders by `offset`. `created_at` comes in 4 formats. `user.role` is missing on register. Errors are only `{ error }`. The 404 body for images is `{ image }`.	`docs/migration/observed/`	The types (§6) model this exactly. The mappers keep components on clean domain types.
A8	**Two API formats.** A `dialect` flag in 7 modules switches between our mock and TechSphere.	§3 table 2	The React app talks to TechSphere only. The mock and the dialect layer stay with the legacy app until cutover (§10).
A9	**Feature gaps against the target routes.** There is no `/checkout` page (checkout is a button in the cart) and no `/admin` page. `account.html` holds both orders and addresses. Home "featured products" are static data (`state/content.js`), not API data.	page inventory, §3	These become new routes and components (§4). Featured products come from the API.

2. Target stack and principles

- **Stack:** React 19, TypeScript (strict, with `exactOptionalPropertyTypes` and `noUncheckedIndexedAccess`), Vite, React Router (data router: `createBrowserRouter`, loaders and actions), Tailwind CSS v4, Vitest with React Testing Library, MSW, Playwright, Storybook with axe-core, and Lighthouse CI.
- **No server-state library.** None is in the allowed tools. Server state is handled by React Router **loaders** (reads, keyed by URL), **actions** (writes, with automatic revalidation), and one small typed `apiClient`.
- **Rules carried over from the legacy app (they work):**
- money in integer cents;
- pure reducers with tests;
- storage access in one module;
- text, never `dangerouslySetInnerHTML`;
- role- and label-based tests;
- per-status error messages;
- token only in the `Authorization` header.
- **Strangler migration.** The React app is built in `web/` next to the legacy pages. Netlify redirects move one route at a time from `*.html` to the React app (`/catalog.html → /products`, `/product.html?id=:id → /products/:id`). The legacy app is deleted at cutover.

3. Folder structure

```
web/                                the React app (wave 0); legacy stays at the root until cutover
├─ index.html                       one HTML shell (replaces 9 pages)
├─ vite.config.ts  vitest.config.ts  playwright.config.ts  tsconfig.json
├─ src/
│  ├─ main.tsx                      createRoot, <RouterProvider>, providers
│  ├─ app/
│  │  ├─ router.tsx                 route objects, lazy() per route, loaders/actions wired
│  │  ├─ providers.tsx              <AuthProvider><CartProvider><Announcer>…
│  │  └─ RootLayout.tsx             <Header/> <main><Outlet/></main> <Footer/> + error boundary
│  ├─ api/                          typed client; no React
│  │  ├─ client.ts                  fetch, timeout, AbortSignal, ApiError, Bearer, 401 hook
│  │  ├─ endpoints.ts               Endpoints map (from types/api.ts)
│  │  └─ mappers.ts                 ApiProduct → Product, ApiOrder → Order … (from api/techsphere.js)
│  ├─ types/                        api.ts (wire), domain.ts (app)  ← drafted in this task
│  ├─ features/                     one folder per feature: route modules + components + hooks + tests
│  │  ├─ catalog/                   ProductsRoute, FiltersPanel, ProductGrid, Pagination, catalog.loader.ts
│  │  ├─ product/                   ProductRoute, Gallery, PurchasePanel, SpecsTable, RecentStrip
│  │  ├─ cart/                      CartRoute, CartLines, OrderSummary, SavedList, CartContext.tsx, cart.reducer.ts
│  │  ├─ checkout/                  CheckoutRoute, checkout.action.ts
│  │  ├─ auth/                      LoginRoute, RegisterRoute, AuthContext.tsx, RequireAuth.tsx
│  │  ├─ account/                   AccountLayout, OrdersRoute, AddressesRoute, loaders/actions
│  │  ├─ admin/                     AdminRoute (summary), RequireRole.tsx
│  │  ├─ home/                      HomeRoute, Hero, CategoryTiles, FeaturedProducts, Deals, Testimonials, Newsletter
│  │  └─ errors/                    NotFoundRoute, RouteErrorBoundary
│  ├─ components/                   shared, presentational, with stories: Price, ProductImage, ProductCard,
│  │                                QuantityStepper, StatusMessage, Button, FormField, ErrorSummary, Skeleton
│  ├─ hooks/                        useAnnounce, useSessionStorage, useDebouncedValue, useDocumentTitle
│  ├─ lib/                          ported pure modules: money.ts, cart-math.ts, dates.ts, query-string.ts,
│  │                                storage.ts, validate.ts, jwt.ts (from src/utils, src/state)
│  └─ styles/                       index.css (@import "tailwindcss"), theme.css (generated from tokens)
└─ tests/
   ├─ unit/        lib/ and reducers (ported Phase 2 unit tests)
   ├─ component/   RTL + MSW (ported tests/dom)
   └─ e2e/         Playwright specs (ported tests/e2e/specs)
```

Rules (ESLint `no-restricted-imports`, as today): `lib/` and `types/` import nothing from the app; `api/` imports only `lib/` and `types/`; `components/` never import `features/`; features import each other only through their `index.ts`.

4. Routing map and component tree

Route	Component (route module)	Data	Guard	Legacy source
`/`	`HomeRoute`	loader: `GET /products?sort=rating` (featured), `GET /categories`	—	`index.html`
`/products`	`ProductsRoute`	loader keyed by search params: `GET /products?…`, facets	—	`catalog.html`
`/products/:id`	`ProductRoute`	loader: `GET /products/:id` (404 → `NotFound`)	—	`product.html?id=`
`/cart`	`CartRoute`	CartContext; loader re-checks prices `GET /products/:id` per line	—	`cart.html`

Route	Component (route module)	Data	Guard	Legacy source
`/checkout`	`CheckoutRoute`	action: `POST /orders`	RequireAuth (back to `/checkout` after sign-in); empty cart → `/cart` with message	cart.html "Place order"
`/login`	`LoginRoute`	action: `POST /auth/login`	signed in → `/account/orders`	`login.html`
`/register`	`RegisterRoute`	action: `POST /auth/register`	as `/login`	`register.html`
`/account`	`AccountLayout` (tabs), index → `/account/orders`	—	RequireAuth	`account.html`
`/account/orders`	`OrdersRoute`	loader `GET /account/orders?offset&limit` (?page= in the URL); action: cancel	RequireAuth	account.html orders
`/account/addresses`	`AddressesRoute`	loader `GET /account/addresses`; actions PUT and DELETE	RequireAuth	account.html addresses
`/admin`	`AdminRoute`	loader `GET /admin/summary`	RequireRole('admin'); shopper → 403 page	**new** (the API has it)
`/compare`	`CompareRoute`	static content	—	`comparison.html` (wave 5, §10)
*	`NotFoundRoute`	—	—	`404.html`
`/catalog.html`, `/product.html?id=:id`, `/cart.html`, `/login.html`, `/register.html`, `/account.html`, `/comparison.html`	301 redirects (Netlify `_redirects`)	—	—	old bookmarks and links

Every legacy feature mapped (to a route and a component):

Legacy feature	Legacy module	Route	React component
Skip link, menu, mobile menu toggle (aria-expanded, Esc)	`nav.js`	all	`Header` → `MainNav`, `MenuToggle`
Cart badge "Cart, n items" (cross-tab)	`cart-badge.js`	all	`Header` → `CartLink` (CartContext)
"Sign in" / "Account" link	`account-link.js`	all	`Header` → `AccountLink` (AuthContext)
Footer	HTML ×9	all	`Footer`
Hero, category tiles	`index.html`	`/`	`Hero`, `CategoryTiles`
Featured products with filter buttons	`catalog-render.js`, `product-filter.js`, `product-card.js`	`/`	`FeaturedProducts` → `FilterButtons`, `ProductCard`
Launch-offers strip (scroll snap, prev/next)	`deals.js`, `scroll-regions.js`	`/`	`DealsCarousel`
Testimonials	`catalog-render.js`	`/`	`Testimonials`
Newsletter form (validation, success)	`newsletter.js`	`/`	`NewsletterForm`
Debounced search, filters with counts, sort, pagination, URL state, Back/Forward	`catalog-view.js`, `filter-bar.js`, `pagination.js`, `state/catalog.js`	`/products`	`ProductsRoute` → `SearchBox`, `FiltersPanel` (`RadioFacet`, `PriceRange`, `RatingSelect`), `SortSelect`, `ProductGrid`, `Pagination`
Loading skeletons, empty state, per-status errors with retry, 429 countdown	`product-list.js`, `status-message.js`	`/products`, `/products/:id`	`ProductGrid` (Skeleton), `EmptyState`, `StatusMessage`
Fault-injection demo panel (#demo)	`demo-controls.js`	—	**not migrated** (§10): MSW stories replace it
Gallery, stock status, quantity stepper, add to cart, save for later, specs table	`product-detail.js`, `gallery.js`, `quantity-stepper.js`	`/products/:id`	`ProductRoute` → `Gallery`, `PurchasePanel` (`QuantityStepper`), `SpecsTable`
Recently viewed strip	`recent-strip.js`, `state/recent.js`	`/products/:id`, `/cart`	`RecentStrip` (`useRecentlyViewed`)

Legacy feature	Legacy module	Route	React component
Cart lines, totals, shipping rule, clear cart, live announcements	`cart-view.js`, `cart-panel.js`	`/cart`	`CartRoute` → `CartLines` → `CartLine`, `OrderSummary`
Saved for later, move to cart	`saved-list.js`	`/cart`	`SavedList` → `SavedItem`
Price and stock re-check on load	`cart-view.js`	`/cart`	`CartRoute` loader + `cart/repriceItems` dispatch
Place order, empty-cart stop, sign-in detour	`checkout.js`	`/checkout`	`CheckoutRoute` → `CheckoutReview`, `PlaceOrderButton`
Sign-in and register forms (validation, 400/401)	`auth-form.js`, `form-errors.js`	`/login`, `/register`	`LoginRoute`, `RegisterRoute` → `FormField`, `ErrorSummary`
Session, expiry redirect, return page	`auth-session.js`, `state/session.js`	all	`AuthProvider`, `RequireAuth`, `useSessionExpiry`
Orders with pages and cancel	`order-list.js`, `account-view.js`	`/account/orders`	`OrdersRoute` → `OrderList`, `OrderRow`, `Pagination`
Address book (edit, delete)	`address-book.js`	`/account/addresses`	`AddressesRoute` → `AddressCard`, `AddressForm`
Sign out (cart kept)	`account-view.js`	all	`AccountMenu` → `SignOutButton` (AuthContext)
Comparison table, "Highlight differences"	`comparison-table.js`	`/compare`	`CompareRoute` → `ComparisonTable` (wave 5)
Admin summary	— (none)	`/admin`	`AdminRoute` → `SummaryCards`
Not found	`404.html`	`*`	`NotFoundRoute`

Component tree:

```
<RouterProvider>
└ RootLayout                          errorElement: RouteErrorBoundary
   ├ AuthProvider                    (Context: session, signIn, signOut)
   │  └ CartProvider                 (Context + useReducer: cart and saved, persisted)
   │     └ Announcer                 (Context: one polite live region)
   │        ├ SkipLink
   │        ├ Header — Logo · MainNav · MenuToggle · CartLink · AccountLink
   │        ├ <main> <Outlet/>
   │        │   ├ HomeRoute — Hero · CategoryTiles · FeaturedProducts(FilterButtons, ProductCard*) · DealsCarousel · Testimonials · NewsletterForm
   │        │   ├ ProductsRoute — SearchBox · FiltersPanel(RadioFacet×2, PriceRange, RatingSelect, ClearFilters) · SortSelect · ResultsStatus · ProductGrid(ProductCard* |
Skeleton* | EmptyState | StatusMessage) · Pagination
   │        │   ├ ProductRoute — Breadcrumb · Gallery · PurchasePanel(Price, StockBadge, QuantityStepper, AddToCart, SaveForLater) · SpecsTable · RecentStrip
   │        │   ├ CartRoute — CartLines(CartLine*: ProductImage, QuantityStepper, Price) · OrderSummary(PlaceOrderLink) · SavedList(SavedItem*) · RecentStrip
   │        │   ├ CheckoutRoute (RequireAuth) — CheckoutReview · PlaceOrderButton · StatusMessage
   │        │   ├ LoginRoute / RegisterRoute — AuthForm(FormField*, ErrorSummary)
   │        │   ├ AccountLayout (RequireAuth) — AccountTabs · <Outlet/>
   │        │   │   ├ OrdersRoute — OrderList(OrderRow*: CancelOrderButton) · Pagination
   │        │   │   └ AddressesRoute — AddressCard*(AddressForm)
   │        │   ├ AdminRoute (RequireRole admin) — SummaryCards
   │        │   ├ CompareRoute — ComparisonTable
   │        │   └ NotFoundRoute
   │        └ Footer
```

5. State-strategy matrix

Rule: state lives in the narrowest place that works.

- **URL:** if it should survive reload or be shared.
- **Server state** (router loader or action): if the server owns it.
- **Context:** only if unrelated parts of the tree need it *and* it changes while the app runs.
- **Local state:** everything else.

Feature / data	Placement	Why here (and why not Context)	Persistence
Catalog search, filters, sort, page	**URL** (`useSearchParams`)	Bookmarkable and shareable; Back/Forward must work (legacy behaviour, tested). Context would lose it on reload.	URL
Search box text before the debounce	**Local** (`useState` + `useDebouncedValue`)	Only the input cares about each keystroke; the URL gets the settled value.	—
Product list, facet counts	**Server** (loader keyed by URL)	The server owns it. Loaders cancel stale requests on navigation (replaces `latest.js`).	router cache
Product detail	**Server** (loader)	Same; a 404 becomes the route's error element.	—
Gallery image, quantity on the product page, mobile menu open, highlight-differences toggle	**Local**	Used by one component; resets when you leave.	—
Cart (lines, quantities)	**Context + useReducer** (`CartProvider`)	Read by the header badge on every route and written from product, cart and checkout. The reducer is the legacy `state/cart.js` ported unchanged (162 tests).	localStorage `brandstore:cart:v1` (same key and shape: legacy and React pages share it during the strangler)
Saved for later	**Same CartProvider** (second slice)	"Move to cart" and "Save for later" change both lists in one step. One reducer makes the move atomic; legacy needed two `set` calls.	localStorage `brandstore:saved:v1`
Cart totals, badge count	**Derived** (`useMemo` over the cart)	Never stored, so the badge and the summary cannot disagree (legacy `cartSummary`).	—
Price and stock re-check	**Server** (cart route loader), then a `reprice` dispatch	Fetched data that updates the cart once.	—
Recently viewed	**Local hook** (`useRecentlyViewed` over sessionStorage)	Written on the product page and read on two pages, never at the same moment. No live updates needed, so no Context and no re-renders across the tree.	sessionStorage `brandstore:recent:v1`
Session (token, user, expiry)	**Context** (`AuthProvider`)	Needed by the header, route guards, the API client (Bearer) and checkout. Changes at sign-in, sign-out and expiry.	sessionStorage `brandstore:session:v1`
Notice and return page after sign-in	**Router state** (`navigate('/login', { state: { from, notice } })`)	Belongs to one navigation, not the app. This replaces the `brandstore:flags:v1` storage key.	none (key retired)
Orders, addresses, admin summary	**Server** (loaders; actions for cancel, PUT, DELETE)	Server data; actions revalidate the loader after a write, so there is no hand-written refresh. Orders page in the URL (`?page=2` → `offset=5`).	—
Form fields and errors (login, register, address, newsletter)	**Local** (`useState`) + router action for submit	Belongs to one form. The action returns field errors (400/401) to `useActionData`.	—
Place order (submitting, result)	**Server** (action) + `useNavigation().state`	The result is the server's; "Placing order…" comes from the router's pending state.	—
Live announcements	**Context** (`Announcer`, `useAnnounce()`)	Seven modules write to their own live regions today (A3). One polite region at the root is announced reliably and is written from anywhere.	—
Request errors, 404	**Route error boundary** (`errorElement`)	Errors thrown in loaders reach the nearest boundary, which renders `StatusMessage` with retry.	—

Context count: 3 (Auth, Cart, Announcer). **Not Context:** catalog filters (URL), products, orders and addresses (server), recently viewed (a hook), sign-in notice (router state), all form and UI toggles (local).

6. TypeScript domain types

Two layers, both in this branch and checked by `npm run typecheck` in CI:

- **`src/types/api.ts`, the wire format.** It is written from 38 captured responses, and `docs/migration/observed-fields.md` gives the field-by-field evidence.

- A field present in every record is required. One that is sometimes missing is optional (`stock_status?` on 22 of 24 products; `role?` missing on register). A field with two JSON types is a union (`ApiPrice = number | string`).
- Closed sets are literal unions: `CategorySlug`, `StockStatus`, `OrderStatus`, `Role`, `ProductSort`, `Currency = 'EGP'`.
- The envelopes differ, as the API does: `ProductPage { items, page, total_pages, total }` and `OrderPage { items, offset, limit, total }`.
- `PlaceOrderRequest.items` is a non-empty tuple, because the server answers 400 for an empty list.
- `Endpoints` maps `"METHOD /path"` to query, body and response, for the typed client.
- **`src/types/domain.ts`, what components use.** The interfaces are `Product`, `Cart`, `CartItem`, `Order`, `User` and `Address`, plus `AuthSession`, `CartTotals`, `Stock`, `Paged<T>`, `Loadable<T>` and `AppError`.
- Money is `Cents`. A product always has at least one image (the placeholder).
- `User.role` is required: the mapper defaults it to `"shopper"`.
- `Order.createdAt: Date | null` keeps `createdAtText`.
- Each field notes the API field it comes from.

How "exactly" is enforced:

1. `scripts/gen-observed-types-check.mjs` writes `src/types/__checks__/observed.ts`. Each of the 38 captured bodies is assigned to its type as an object literal. TypeScript's excess-property and missing-property checks then fail on any field the types miss, require wrongly or mistype. This was tried: making `stock_status` required gives 5 errors, one for each captured response that contains p-007 or p-018.
2. `src/types/__checks__/strictness.ts` checks the other direction. Fifteen `@ts-expect-error` lines prove the types reject what the API never sends: a null price, `stock_status: 'unknown'`, a string order total, an `offset` on a product page, an empty order.

API field (wire)	Domain field	Mapping (ported from `api/techsphere.js`)
`price: number | string`	`price: Cents`	`priceToCents`: strip to digits, ×100, round; unreadable → error, never 0
`stock_status?`	`stock: { status, available }`	missing → `'unknown'`, treated as in stock with 25 (HANDOFF gap 2)
`reviews_count`	`reviewCount`	rename
`images: string[]` (JSON endpoint)	`images: [ProductImage, …]`	absolute URLs kept; root-relative paths → placeholder
`specs: Record<string, string>`	`specs: Spec[]`	readable labels, empty values dropped
`created_at` (4 formats)	`createdAt: Date | null` + `createdAtText`	`parseOrderDate` (ported `utils/dates.js`)
`total` (pounds)	`total: Cents`	×100
`user.role?`	`role`	default `'shopper'`
`{ page, total_pages }` / `{ offset, limit }`	`Paged<T>`	page size fixed at 8 for products; offset → page for orders

7. Tailwind CSS adoption with the brand tokens

- **One source of truth.** The 105 tokens in `css/base.css :root` are converted by `scripts/tokens-to-tailwind.mjs` into a Tailwind v4 `@theme` (`docs/migration/tailwind-theme.css`, mapping in `tailwind-tokens.md`):
- 68 tokens land in Tailwind namespaces. Colours are `--color-*` already, so `bg-primary` and `text-text-muted` read like the legacy names. `--space-*` becomes `--spacing-*` (`p-md`), and `--weight-*` becomes `--font-weight-*`. Text, radius, shadow, leading, tracking and ease keep their names. The breakpoints are 48em, 64em and 90em, as today.
- 37 stay CSS variables, used as `w-(--size-sidebar)` or `z-(--z-header)`: layout sizes, z-index, durations, focus ring.
- **Coexistence.** Legacy pages keep their four CSS files until cutover. The React app imports Tailwind with its own theme only, so there are no shared selectors and no preflight conflicts with `base.css`.
- **Components, not utility soup.** Repeated patterns become React components with Tailwind classes inside: `Button` variants, `FormField`, `Price`, `StatusMessage`. Ad-hoc `@apply` is not used. `prettier-plugin-tailwindcss` sorts classes.
- **Kept behaviour:** focus rings (`focus-visible:outline-(--focus-ring-color)`), `motion-reduce:` variants for the legacy reduced-motion policy, and contrast. Phase 1's lowest ratio was 5.02, and axe-core in Storybook and Playwright checks it.
- **Done when:** each migrated component's Storybook story passes axe, and its Playwright screenshot matches the legacy page at 375 px and 1280 px.

8. Testing strategy

Layer	Tool	What it covers	Phase 2 tests ported
Unit	Vitest	`lib/` (money, cart math, dates, query strings, storage, validate, jwt), reducers, mappers (API → domain)	`tests/unit` (17 files, 162 cases): **moved, then the imports are changed to TS**. They are framework-free, so they become the safety net of wave 0.
Contract	Vitest + captured fixtures, `tsc`	Mappers against `docs/migration/observed/*.json`; types against the same (`npm run typecheck`)	`tests/unit/techsphere.test.js`, `edge-sweep.test.js` → `mappers.test.ts`, driven by the captured responses
Component	Vitest + React Testing Library + MSW (jsdom)	Each route and shared component: 200, empty, 401, 404, 429, 500; announcements; focus	`tests/dom` (4 files, 26 cases) → RTL. MSW handlers and role/label/text queries carry over; `screen.js` is replaced by RTL's `screen`.
API integration	Vitest + MSW	`api/client.ts`: timeout, abort, Bearer, 401 hook, Retry-After	`tests/integration` (21 cases); the BrandStore-mock cases are dropped with the dialect layer
End-to-end	Playwright against the TechSphere API (kit server only when `VITE_API_BASE` is unset)	The journeys: catalog → cart, sign-in with a wrong password → orders, the five edge cases, keyboard-only checkout; Chromium in CI, plus a Firefox and WebKit smoke test	`tests/e2e/specs` (9 specs): **kept**. They query roles and labels, so only URLs change (`/catalog.html` → `/products`). The 28 journeys in `tests/e2e/*.mjs` are retired after parity (§10).
Accessibility	axe-core (Storybook addon and `@axe-core/playwright`)	Every story and every route	Replaces `qa/keyboard.js` and `qa/contrast.js`
Performance	Lighthouse CI with budgets	Per route: LCP ≤ 2.5 s, CLS ≤ 0.1, TBT ≤ 200 ms (mobile); first-load JS ≤ 90 KB gzip; route chunks ≤ 30 KB	Replaces `scripts/lighthouse.mjs`; baseline in §1 A5

Coverage gate: 80% of lines in `lib/` and `api/`, as today for `utils/` and `state/`. Components are covered by RTL behaviour tests, not by a percentage.

Order: port the tests before the code they cover, so every wave begins with red tests and ends green. The legacy suite keeps running against the legacy pages until each one is removed.

9. Migration waves (lowest risk first, each one de-risking the next)

Risk = likelihood (1–3) × impact (1–3). Evidence refers to §1.

Wave	Scope	Risk	Evidence	Exit criteria
0. Foundation	`web/` with Vite, React, TS strict, Tailwind theme, Vitest/RTL/MSW, Playwright, Storybook, CI jobs. Port `utils/`, `state/` reducers and the API mappers to TS (`lib/`, `api/`) with their tests	**2** (L1 × I2)	A6: pure modules at about 99% line coverage; A1: the API boundary is where types pay off	162 unit tests green in TS; `tsc` strict with no errors; CI runs both apps
1. Shell and read-only catalog	`RootLayout`, Header/Footer, `/`, `/products`, `/products/:id`, `*`; `ProductCard`/`ProductImage`/`Price`/`StatusMessage`; redirects for catalog and product	**4** (L2 × I2)	A2/A3: shell and image duplication; the catalog has the most moving parts (URL state, facets, 3 loaders in `catalog-view.js`), but it is read-only	e2e specs for catalog and the edge cases pass on the React routes; Lighthouse budgets met
2. Cart, saved and recent	`CartProvider` (ported reducer), `/cart`, header badge, re-pricing, recent strip	**6** (L2 × I3)	A4: the cart is created on 9 pages; it is money and persistence; the Task 3 grader feedback was about cart totals, quantity bounds and persistence	Cart shared with the legacy pages through the same key; the cart and live-region specs pass
3. Auth and account	`AuthProvider`, `/login`, `/register`, `RequireAuth`, expiry, `/account/orders`, `/account/addresses`; flags key retired	**6** (L2 × I3)	Security (`docs/SECURITY.md`); offset pagination and 4 date formats (A7)	Sign-in, expiry, leak and orders specs pass; no token in URLs or logs (existing leak test)
4. Checkout and admin	`/checkout` (moved out of the cart), `/admin` with `RequireRole`	**9** (L3 × I3)	A9: a new page and a new flow; `POST /orders` validates nothing and returns no totals (HANDOFF gap 3)	Empty-cart and keyboard checkout specs pass; admin 403 for a shopper
5. Cutover and clean-up	All redirects, delete legacy pages and `src/`, the mock dialect and Axios; `/compare`; retire the journey suite and `qa/` scripts	**3** (L1 × I3)	A5/A8: removes the duplicate client and the dialect flag	One app; Lighthouse CI on every route; README updated

Why this order: each wave reuses what the previous one proved. Wave 0 makes the shared logic typed and tested. Wave 1 proves routing, layout, loaders and the design system on data nobody can lose. The cart (wave 2) and the session (wave 3) are the two Contexts, and they are needed before checkout. Checkout and admin (wave 4) carry the highest risk, so they come last, on a foundation that is already in production.

Estimates

Assumptions:

- One developer at 6 focused hours a day, so an ideal day is 6 hours. Code review the next working day.
- The TechSphere API stays at kit 1.1 (no contract changes).
- No visual redesign: the tokens and markup patterns are reused.
- Tests are ported, not rewritten (§8).
- CI already exists.
- Story points use Fibonacci and are calibrated on Phase 2: the cart (Task 3) was about 8 points and about 5 ideal days.
- 20% contingency on the total.
- Not included: waiting for backend fixes from `docs/HANDOFF.md`.

Wave	Main items (ideal days)	Ideal days	Story points
0	Scaffold and CI 1.0 · port 20 pure modules and 162 tests to TS 2.5 · typed client and mappers 1.5 · Tailwind theme and base components 1.0 · Storybook and axe 0.5	**6.5**	8
1	Layout and shell 1.0 · home 1.5 · catalog (URL state, facets, pagination, states) 3.0 · product page 1.5 · redirects and e2e ports 1.0	**8.0**	13
2	CartProvider and storage sync 1.5 · cart page and summary 1.5 · saved and recent 1.0 · re-pricing 1.0 · announcer and focus 0.5 · tests 1.0	**6.5**	8
3	AuthProvider, guards, expiry 1.5 · login and register actions 1.0 · orders with offset pages and cancel 1.0 · addresses 0.5 · tests 1.0	**5.0**	8
4	Checkout route, action, empty-cart and sign-in paths 2.0 · admin route and role guard 1.0 · tests 1.0	**4.0**	5
5	Redirects and deletion 1.0 · `/compare` 0.5 · Lighthouse CI budgets 0.5 · docs 0.5	**2.5**	3
Total		**32.5** (+20% → **39**)	**45**

At 5 ideal days a week, that is about 8 weeks for one developer. Waves 2 and 3 can run in parallel with two developers, because they meet only in the header: about 6 weeks.

10. What will not migrate now, and why

Item	Decision	Reason	Revisit
`comparison.html` (static comparison table)	Last, in wave 5; stays a legacy page until then	No API, no state, rarely visited; nothing depends on it	Wave 5
Catalog fault-injection panel (`#demo`)	Not migrated	A reviewer tool; MSW stories in Storybook (200, empty, 401, 429, 500) do the same job without shipping it	—
BrandStore mock API (`mock-server/`, `api/openapi.json`, contract tests, Netlify function) and the `dialect` layer	Not migrated; deleted at cutover	The React app talks only to TechSphere (`VITE_API_BASE`, fallback `http://localhost:4010`). Two formats doubled the API code (A8).	—
Axios	Not migrated	A second HTTP client for one page type (17.5 KB gzip, A5); `api/client.ts` covers it	—
The 28-journey suite (`tests/e2e/*.mjs`) and Phase 1 `qa/*.js`	Kept against legacy pages, then retired	Superseded by Playwright specs, axe-core and Lighthouse CI; porting custom runners costs more than it protects	Wave 5
Test stand-ins (`tests/support/browser`, `playwright` shim, `vitest-shim`)	Not migrated	They existed only for an environment without the real packages; CI runs the real tools	—
Server-side rendering or static generation	Not now	Not in the allowed tools; Lighthouse SEO is already 100; the SPA's first-load cost is managed with route splitting and budgets	After cutover, if LCP misses budget
Arabic (`?locale=ar`, RTL)	Not now	The API supports it, but the legacy app does not; adding it doubles visual QA	Separate project
Creating addresses, refresh tokens, order totals	Not now	The API has no endpoint for them (HANDOFF gaps 3, 12, 13)	When the backend ships them
A server-state library (for example TanStack Query)	Not now	Not in the allowed tools; router loaders and actions cover reads, writes and revalidation	If caching across routes becomes a problem

Item	Decision	Reason	Revisit
Visual regression testing as a CI gate	Not now	Screenshots are compared by hand per component in waves 1–4; a full baseline is only stable after cutover	Wave 5

11. Risks and mitigations

Risk	Mitigation
A cart lost or duplicated while legacy and React pages run side by side	Same localStorage keys and shape; the reducer is ported unchanged with its 162 tests; a cross-app Playwright test adds on a legacy page and reads on a React page
First-load JS grows (React, router)	Route-level `lazy()`, budgets in Lighthouse CI, no Axios, no CSS frameworks other than Tailwind
Messy API data breaks components	Components only see domain types; the mappers are tested against the captured responses; `npm run typecheck` fails when the API changes shape (re-run `npm run api:capture`)
Scope creep in "new" routes (checkout, admin)	Wave 4 last, with acceptance criteria fixed now (§4 table)
The grading machine has no Python	`dev`, `test` and `e2e` start without it. Playwright starts the kit only when `VITE_API_BASE` is unset (already true in `playwright.config.mjs`)